FIAP ON

TRATORES DIGITAIS - HARDWARES PARA IA

FUNÇÕES E PORTAIS: DESVENDANDO A CONECTIVIDADE INTELIGENTE

6

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Exemplo de sinal contínuo no tempo em uma janela de 60s 8
Figura 2 - Vários instantes do sinal analógico 10
Figura 3 - Sinal contínuo com 4 períodos de 60s de duração 11
Figura 4 - Amostragem mínima ocorrendo nos pontos vermelhos 12
Figura 5 - Amostras que serão armazenadas no ESP32 12
Figura 6 - Comparativo entre amostras em vermelho que lembrarão da senoide original em azul 13
Figura 7 - Amostragem com mais pontos vermelhos dentro de 1 período do sinal azul.....13
Figura 8 - Figura da amostragem com mais pontos dentro do período 15
Figura 9 - Amostragem da senoide analógico com F_a = 0,4Hz 16
Figura 10 - Etapa da quantização com 3 bits de resolução.....18
Figura 11 - Quantização usando 4 bits de resolução 19
Figura 12 - Etapa de quantização usando 6 bits 19
Figura 13 - Etapa da quantização usando 8 bits de resolução.....20
Figura 14 - Etapa de quantização usando 12 bits de resolução.....20
Figura 15 - Circuito 1 com um potenciômetro e um LED usando ADC 26
Figura 16 - Monitor Serial do Wokwi mostrando as impressões do exemplo 1 28
Figura 17 - Dados impressos no Monitor Serial do exemplo 2 30
Figura 18 - Circuito do ESP32 coletando dados por meio de dois potenciômetros...34
Figura 19 - Ambiente Colab rodando o treinamento em Python.....37
Figura 20 - Dica para abrir o modo gráfico do Monitor Serial 42
Figura 21 - Senoide sendo reproduzida pelo GPIO 25.....42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE TABELAS

Tabela 1 - Explicação de como o codificador trabalha para codificar um dado pico .23
Tabela 2 - Exemplo de critérios de rotulagem que serão feitas pelo ESP32.....32
Tabela 3 - Exemplo de rotulação de “saúde” (1) e “risco” (1).....35

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de como ADC trabalha no ESP32 27
Código-fonte 2 - Segundo exemplo de ADC, mas agora usando média móvel.....29
Código-fonte 3 - Exemplo de uso de ADC para um sistema de saúde 34
Código-fonte 4 - Código de treino do modelo de saúde e risco.....36
Código-fonte 5 - Teste do modelo inserindo dados manualmente 38
Código-fonte 6 - Exemplo de código para reconstituir o sinal senoidal que estava digital.....41
Código-fonte 7 - Exemplo de sistema de saúde com dois tons distintos, um para “saúde” e outro para “risco”.....45

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 FUNÇÕES E PORTAIS: DESVENDANDO A CONECTIVIDADE INTELIGENTE ..6
1.1 Conversores de sinais no ESP32 6
1.1.1 Introdução 6
2 CONVERSORES DE SINAIS NO ESP32 8
2.1 Conversor ADC 9
2.1.1 Amostragem 9
2.1.2 Quantização 16
2.1.3 Codificação.....21
2.2 Códigos e Aplicações para ADC 24
2.2.1 Primeiro exemplo para ADC.....24
2.2.2 Segundo exemplo de ADC 28
2.2.3 Terceiro exemplo de ADC 30
2.3 Conversor DAC 39
2.3.1 Exemplo de DAC.....41
CONCLUSÃO.....46
REFERÊNCIAS.....48

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 FUNÇÕES E PORTAIS: DESVENDANDO A CONECTIVIDADE INTELIGENTE

1.1 Conversores de sinais no ESP32

No ESP32, existem dois tipos de conversores, o Conversor Analógico-Digital (ADC) e o Conversor Digital-Analógico (DAC). Ambos desempenham um papel muito importante na integração de sensores e atuadores em projetos de IA e, ao longo desse capítulo, você vai entender o motivo.

1.1.1 Introdução

Os componentes ADC e DAC são internos e nativos de qualquer microcontrolador, e permitem que o ESP32 interprete sinais do mundo físico e interaja com ele, conectando o ambiente real aos sistemas computacionais. Não se trata de sensores, mas sim, de um conversor que converte um sinal analógico do ambiente externo para digital (ADC), que por sua vez, será tratado pela arquitetura digital do ESP32. E quando esse sinal digital tratado internamente pela arquitetura do ESP32 estiver pronto, ele será externado de volta ao ambiente analógico de onde ele partiu, convertendo-o de digital para analógico (DAC).

Em projetos de IA, onde a coleta e o processamento de dados do ambiente são essenciais para a tomada de decisões, o ADC converte sinais analógicos a partir de sensores analógicos, como variações de temperatura, pressão ou luminosidade, em valores digitais utilizáveis. Por outro lado, o DAC transforma informações digitais em sinais analógicos, possibilitando o controle de dispositivos como motores e alto-falantes, que são dispositivos analógicos por natureza.

Essa interação entre hardware e software, mediada pelos conversores, é comum para aplicações de IA que exigem precisão e confiabilidade na análise de dados.

Com sua capacidade de lidar com múltiplos canais ADC, isto é, possui diversos ADC) e produzir saídas analógicas pelo DAC, o ESP32 permite a construção de sistemas embarcados inteligentes e eficientes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Projetos como agricultura de precisão, monitoramento de saúde, sistemas industriais e automação residencial utilizam essas capacidades para capturar dados em tempo real e alimentar algoritmos de Inteligência Artificial, contribuindo para soluções mais adaptáveis e inteligentes.

Com isso, o ESP32 se torna uma plataforma atraente e acessível (o custo-benefício é muito bom) no desenvolvimento de sistemas integrados que conectam o mundo físico ao digital, ampliando o impacto da IA em diversos setores.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 CONVERSORES DE SINAIS NO ESP32

Nesse primeiro momento, é importante você saber o motivo de precisarmos converter o sinal analógico para digital para trabalhar com o ESP32. Não seria possível ter menos dor de cabeça e trabalhar com sinais analógicos da forma que surgiram no mundo? Infelizmente, não!

Sinais analógicos são contínuos, variando suavemente ao longo do tempo no eixo vertical Y (eixo das amplitudes) à medida que o tempo passa no eixo horizontal X (eixo dos tempos), pois a tensão é gerada por sensores de temperatura, pressão ou luminosidade. No eixo Y, tem-se os valores da amplitude do sinal variando enquanto o tempo passa, seja em qual for a unidade (segundos, minutos, horas). Veja o gráfico a seguir.

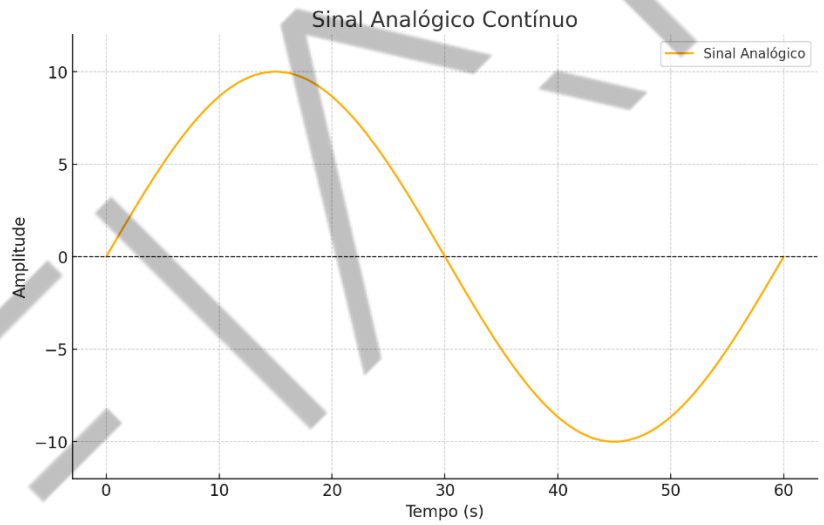

Figura 1 - Exemplo de sinal contínuo no tempo em uma janela de 60s
Fonte: Elaborado pelo autor (2024)

Então, se esses sinais refletem as mudanças reais no ambiente físico a cada instante que passa, o problema é que esse sinal contínuo na amplitude e no tempo é incompatível com qualquer chip digital, pois como o próprio nome diz, o chip sendo digital só entende dois valores, o ZERO e o UM, e não uma variedade de valores entre ZERO e um valor X ou Y.

Os sinais digitais são representados por valores discretos (números binários), usados por dispositivos eletrônicos para realizar cálculos, armazenar informações e tomar decisões.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Portanto, essa incompatibilidade entre sinal e ESP32 é resolvida quando os dados são convertidos em uma forma compreensível ao sistema digital, ou seja, traduzidos em números binários por meio do ADC.

2.1 Conversor ADC

Resgatando o conceito desse processo de conversão, toda vez que um sinal analógico precisar ser convertido para o mundo do digital, isto é, aplicar o ADC, ele obrigatoriamente deverá passar por essas três etapas:

- Amostragem;
- Quantização;
- Codificação.

2.1.1 Amostragem

A amostragem consiste em capturar “fotos” do sinal analógico em intervalos regulares de tempo. Não é obrigatório que o intervalo de tempo entre cada foto capturada esteja igual ao intervalo de tempo original do sinal. Na verdade, o intervalo de tempo entre cada foto capturada precisa ser mais rápido que o intervalo de tempo necessário para o sinal analógico completar um período.

O que é um período do sinal analógico? É o intervalo de tempo que o sinal analógico original gasta para se apresentar em sua forma inédita, sem repetir a sua periodicidade. Vamos analisar o gráfico da figura “Vários instantes do sinal analógico”.

Na primeira curva da imagem, o sinal está compreendido entre 0 e 15s e sua amplitude foi de 0 a +10. Esse sinal não se apresentou como todos os pontos de sua forma inédita, porque ainda não tivemos uma repetição de algum valor em amplitude, ou seja, todos os valores apresentados de 0 a 10s são inéditos.

Desse modo, podemos observar mais instantes futuros, isto é, mais alguns segundos para conferir se algum ponto em amplitude se repete como o original. A segunda curva da imagem, compreendida de 0 a 30s, não repetiu nenhum ponto em amplitude, portanto, não chegamos a conhecer o período desse sinal.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

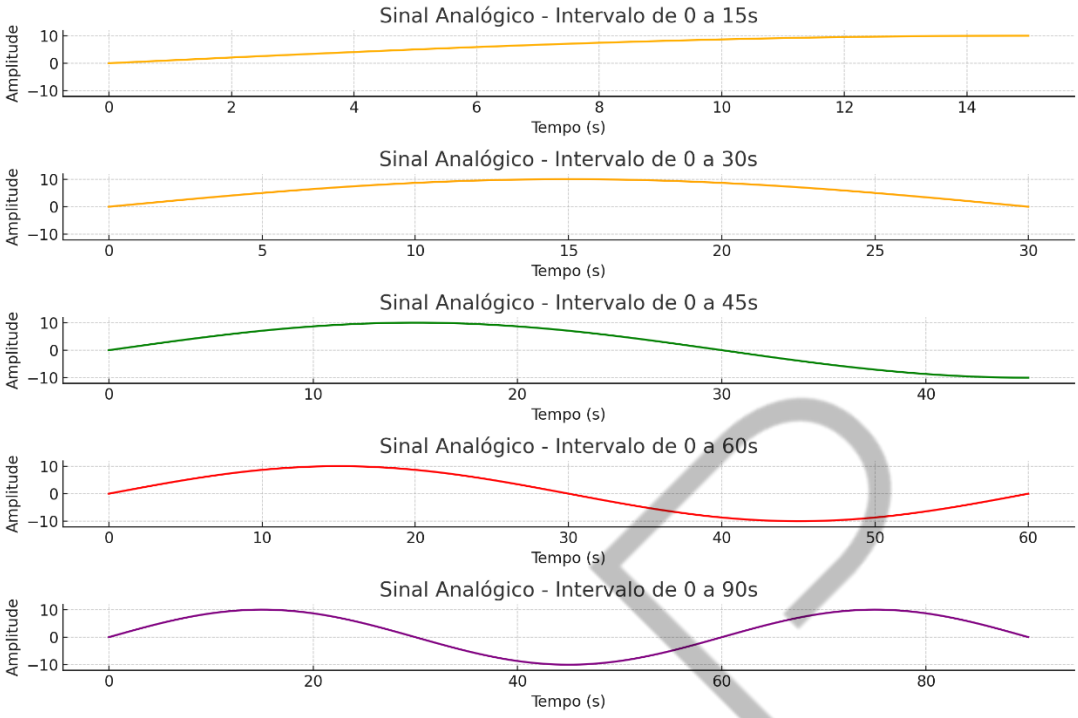

Figura 2 - Vários instantes do sinal analógico
Fonte: Elaborado pelo autor (2024)

Na terceira curva (em verde), o sinal analógico ainda continua se apresentando com seus pontos inéditos, agora com valores negativos na amplitude (de 0 à -10) e no eixo X, de 0 à 45s. Ainda assim, não temos o período todo da onda porque o sinal não repetiu a sua forma em nenhum ponto de X e Y ao longo de sua trajetória.

Na quarta curva (em vermelho), o sinal analógico voltou para a amplitude 0 no instante 60s depois de atingir o -10 no instante 45s. Isso é uma indicação que **o sinal analógico pode estar se apresentando como um período completo** compreendido de 0 a 60s porque ele começa a repetir a sua trajetória a partir dos 60s, iniciando um novo período. E sim, esse sinal possui um período de 60s porque, ao deixar mais tempo na sua excursão, notamos que a cada 60s ele se repete por inteiro, subindo até +10, passando por 0, descendo até -10 e retornando até 0 dentro de 60s, se repetindo depois. Veja o gráfico a seguir para um melhor entendimento.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

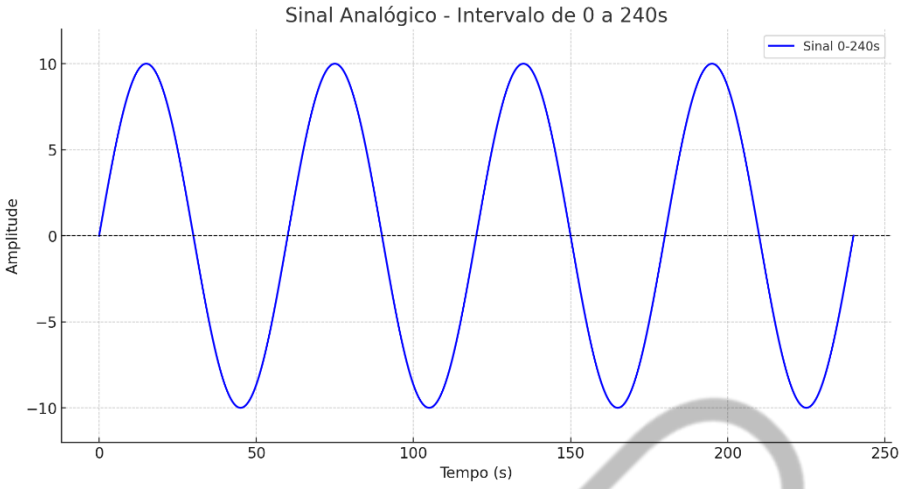

Figura 3 - Sinal contínuo com 4 períodos de 60s de duração
Fonte: Elaborado pelo autor (2024)

Mas por que precisamos realizar toda essa análise do período? Para que você entenda quantas “fotos” minimamente tirar na etapa (1) Amostragem da digitalização do sinal analógico original.

Entendido o que é 1 período do sinal, **a quantidade mínima de fotos que devemos tirar do sinal analógico é de duas fotos** dentro desse 1 período. O instante das fotos é sempre equidistante entre si. Na figura “Amostragem mínima ocorrendo nos pontos vermelhos”, deixamos em destaque vermelho os instantes equidistantes onde haveria uma “foto” do meu sinal analógico.

Esses pontos vermelhos são minimamente necessários para o sistema “lembrar” que o sinal original já foi uma senoide, pois com esses pontos vermelhos gravados na memória do ESP32, ele seria capaz de reconstituir a senoide. Basta ele saber qual era a amplitude máxima e mínima (ou seja, +10 e -10) e reconectar os pontos vermelhos subindo e descendo a cada “montanha” da senoide. E assim, não perderíamos informações do sinal analógico original.

Afinal, “amostrar” significa tirar amostras, isto é, tirar fotos, e descartar todo o restante dos pontinhos contínuos em azul do sinal original.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

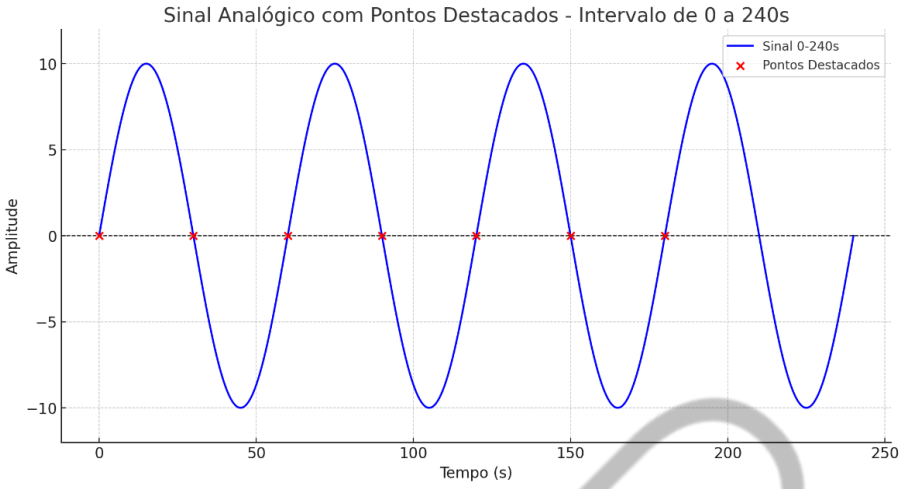

Figura 4 - Amostragem mínima ocorrendo nos pontos vermelhos
Fonte: Elaborado pelo autor (2024)

Então, o resultado do processo de amostragem é o armazenamento de amplitudes zero em intervalos de tempo, como mostra a figura “Amostras que serão armazenadas no ESP32”. Além disso, a figura também informa a quantidade mínima de fotos no processo de amostragem que garante a recuperação do sinal analógico senoidal original.

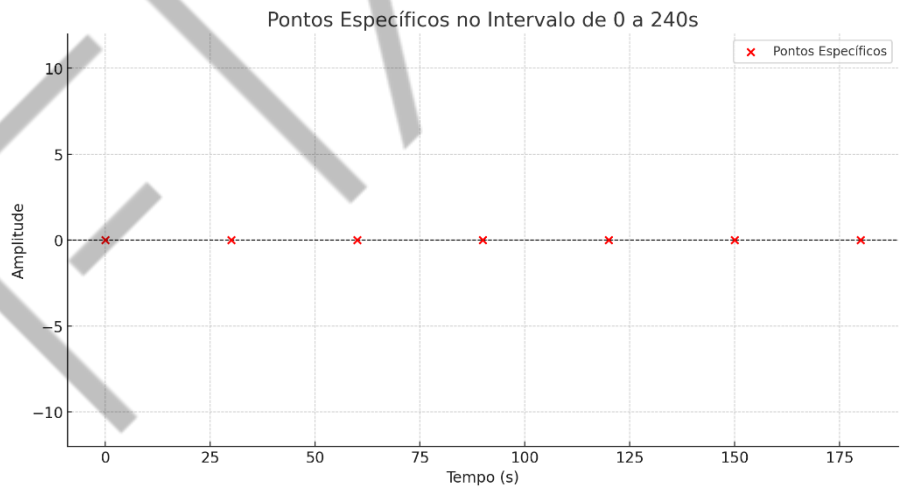

Figura 5 - Amostras que serão armazenadas no ESP32
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

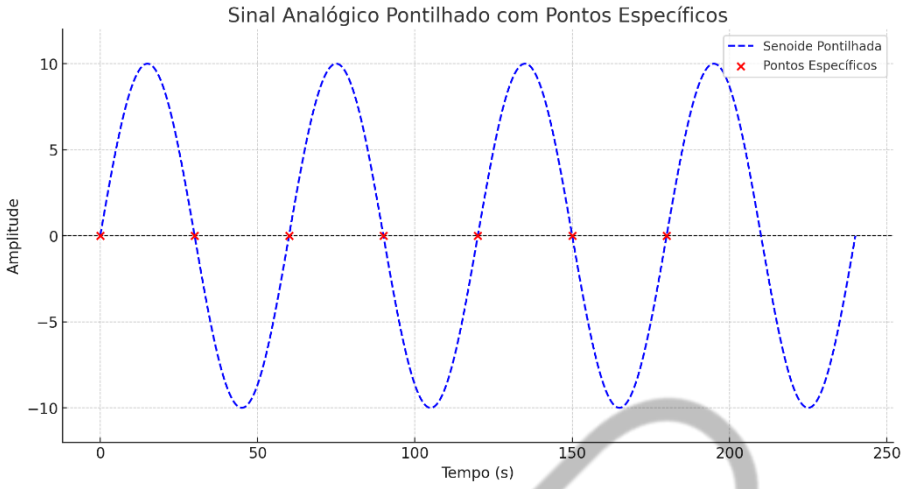

Figura 6 - Comparativo entre amostras em vermelho que lembrarão da senoide original em azul
Fonte: Elaborado pelo autor (2024)

Porém, tirar duas fotos a cada período da senoide não é uma regra fixa e sim, o mínimo de fotos possível para que recuperemos a senoide mais tarde, quando formos desfazer o processo de digitalização.

Veja o gráfico a seguir que tem três fotos por período:

1. A primeira foto foi tirada no instante zero.
2. A segunda foto no instante 15s.
3. A terceira no instante 45s.
4. E na quarta foto, em 60s, mas nesse instante já começa outro ciclo do sinal, então, não conta como o do primeiro ciclo, e sim, do segundo ciclo.

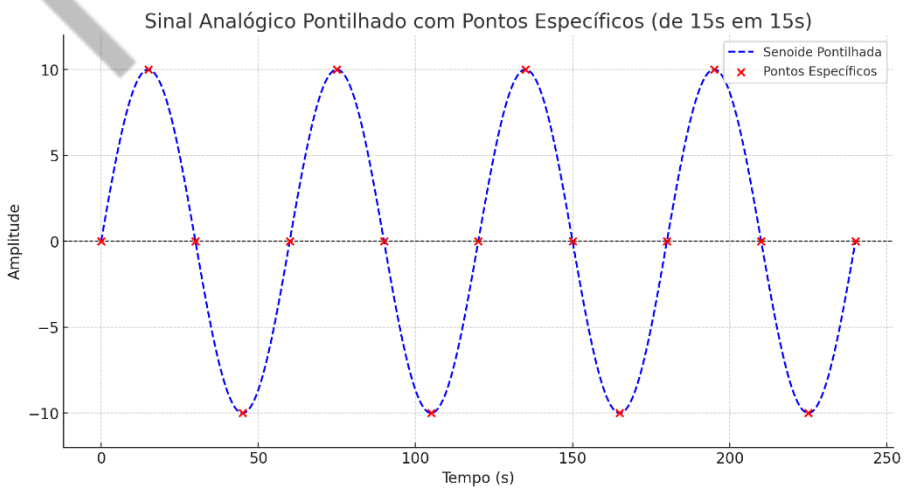

Figura 7 - Amostragem com mais pontos vermelhos dentro de 1 período do sinal azul
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Essa regra de tirar no mínimo duas fotos dentro do período do sinal chama-se Teorema da Amostragem de Nyquist-Shannon. Este teorema estabelece que, para representar um sinal analógico de forma precisa após a amostragem, o intervalo de tempo entre as amostras deve ser pelo menos a metade do intervalo de um período do sinal original. Então, se o período original do nosso gráfico possui 60s, as fotos da amostragem devem ocorrer no mínimo a cada 30s. Entretanto, vimos nos gráficos anteriores que temos fotos tiradas a cada 15s, que é metade da metade do período do sinal. E isso está em conformidade com o Teorema de Nyquist-Shannon.

Portanto, todas as vezes que você for digitalizar um sinal analógico, e convertê-lo para digital para ser armazenado dentro de um microcontrolador, tenha em mente a velocidade ou a frequência das fotos sobre o sinal original.

Ou você pensa no domínio do tempo:

$$Tamostragem \leq Tempo\ do\ período\ /\ 2$$

Ou você pode pensar no domínio da frequência:

$$Famostragem \geq 2\ x\ Frequência\ do\ período$$

No exemplo do sinal dos gráficos, possui 60s de período. Então: $Tamostragem \leq 60\ /\ 2$

Isto é: $Tamostragem \leq 30$

Ou: $T = 1/F$

Logo: $60s = 1\ /\ F$ (passando o 60 para o denominador e subindo o F para o numerador).

Então: $F = 1/60$

$F = 0{,}01667Hz$

Pela regra de Nyquist: $Famostragem \geq 2\ x\ Frequência\ do\ período$

$Famostragem \geq 2\ x\ 0{,}0166Hz$

$Famostragem \geq 0{,}0333Hz$

Qualquer valor acima de 0,033Hz atende os requisitos da amostragem ideal.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Considerando a figura “Amostragem com mais pontos vermelhos dentro de 1 período do sinal azul”, a frequência de amostragem é a seguinte:

$Tamostragem = 15s$ (a cada 15s tem uma amostra)

$F = 1 / T$

$F = 1 / 15s$

$F = 0{,}666Hz$ (frequência da amostragem convertida em Hz)

Pela regra de Nyquist: $Famostragem \geq 2\ x\ Frequência\ do\ período$

$Famostragem \geq 2\ x\ 0{,}666Hz$

$Famostragem \geq 0{,}133Hz$

Note que essa frequência está bem maior que a de 0,0333Hz, justamente porque essa última tem mais pontos de amostragem dentro do mesmo período do sinal.

E o resultado dessa última amostragem está no gráfico a seguir.

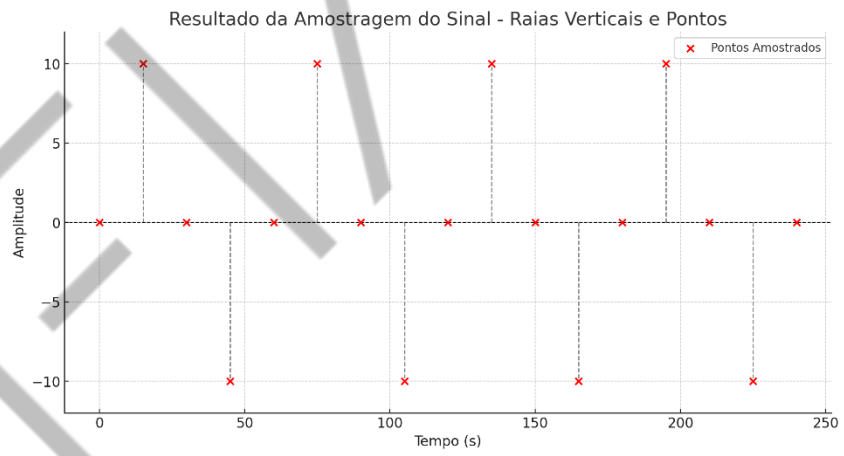

Figura 8 - Figura da amostragem com mais pontos dentro do período
Fonte: Elaborado pelo autor (2024)

Na figura “Amostragem da senoide analógico com F_a = 0,4Hz”, temos o intervalo de 5s entre cada amostra (ou 1/5s = 0,02Hz), que resulta em: $Famostragem = 0{,}4Hz$ (porque 2 x $F_{\text{frequência do período}}$).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

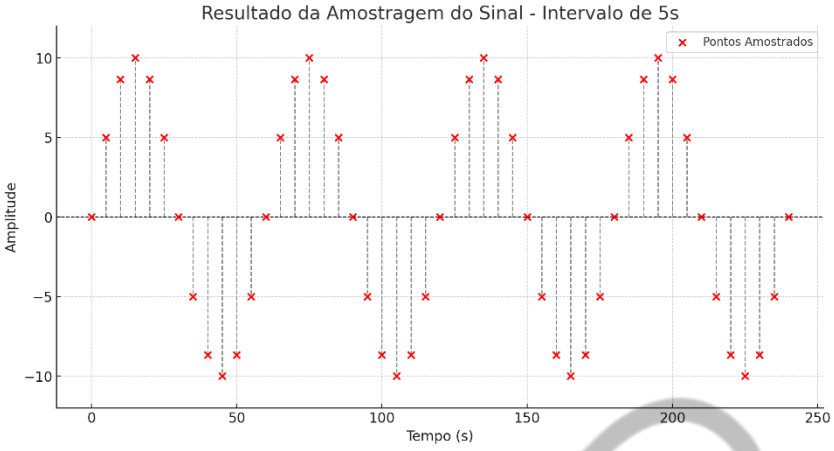

Figura 9 - Amostragem da senoide analógico com F_a = 0,4Hz
Fonte: Elaborado pelo autor (2024)

Para encerrar, note que nessa última amostragem, percebe-se nitidamente a formação de uma senoide, porém “semidigital”. Quanto mais pontos tiver, mais memória será consumida para armazenar no microcontrolador. Portanto, fique atento ao ponto de equilíbrio para não consumir memória demais.

Infelizmente, não existe regra para limitar o máximo de pontos, mas sim para o mínimo de pontos (Nyquist), para que seja possível reconstituir o sinal mais tarde quando ele retornar para o mundo analógico.

2.1.2 Quantização

Relembrando também, a **quantização** é um processo primordial na conversão de sinais analógicos para digitais, **utilizado após a etapa de amostragem**. Consiste em mapear os picos das amostras provenientes dos valores contínuos do sinal analógico para um conjunto finito de níveis discretos.

Isso é realizado porque os sistemas digitais, como o ESP32, não conseguem lidar com infinitas possibilidades de valores de amplitudes; em vez disso, eles representam os sinais analógicos como números digitais com uma precisão limitada definida pela resolução do conversor analógico-digital (ADC).

No caso do ESP32, o ADC possui uma resolução de 12 bits, isso significa que o intervalo de valores possíveis na entrada, geralmente de 0 a 3,3V para o ESP32, é dividido em 2^{12} = 4096 níveis discretos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Cada nível corresponde a um pequeno degrau de intervalo que os picos das amostras podem se arredondar. Durante o processo de quantização, o valor analógico lido é aproximado para o nível mais próximo, resultando em uma representação digital que pode ser processada pelo microcontrolador.

A quantização é importante no mundo da IA, especialmente ao utilizarmos dispositivos como o ESP32, pois ela define a qualidade dos dados capturados pelos sensores **analógicos** (sensores digitais já entregam o sinal digitalmente). Esses dados quantizados são frequentemente utilizados como entradas para modelos de IA, influenciando diretamente sua precisão e desempenho.

Se o processo de quantização não for bem ajustado, como em casos de baixa resolução, pode ocorrer perda significativa de informações, reduzindo a eficácia do modelo.

Além disso, no contexto de IA embarcada, a quantização também se aplica a modelos de aprendizado de máquina. Técnicas como quantização de pesos e ativação em redes neurais reduzem a precisão dos números usados nos cálculos do modelo, permitindo que ele seja executado em dispositivos com recursos limitados, como o ESP32.

Isso é importante para viabilizar a inferência em tempo real diretamente no microcontrolador, otimizando o uso de energia e memória sem comprometer significativamente o desempenho.

Vamos aos gráficos para melhor entender a quantização. Na figura a seguir, temos um exemplo de quantizador utilizando 3 bits de resolução. Ou seja, 3 bits dariam 2^{3bits} = 8 níveis ou 8 degraus. Esses 8 níveis são divididos sempre assim: a metade deles fica na parte negativa do gráfico sem contar o nível zero (eixo X propriamente dito), e a outra metade -1 fica na parte superior do gráfico. O desconto de -1 da parte superior é porque o nível zero entra na metade da parte superior.

Note os picos em vermelhos originais da etapa anterior, ou seja, da Amostragem, sendo arredondados para o nível mais alto que é entre 5 e 10 (7,5). Esse arredondamento para baixo, de 10 para 7,5 é um erro chamado Erro de Quantização.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O erro de quantização foi pior na parte positiva porque três picos de amostras em vermelho tiveram que retornar para 7,5, enquanto na parte negativa do gráfico, dois picos tiveram que retornar para -7,5.

Esse erro de quantização alterou o valor de pico original da senoide. Agora a senoide está com os picos “achatados”. Isso é o que acontece quando temos baixa resolução no processo de quantização.

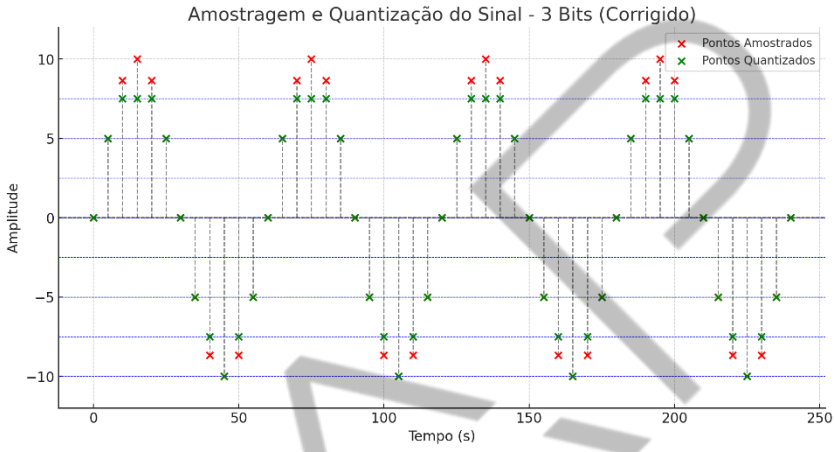

Figura 10 - Etapa da quantização com 3 bits de resolução
Fonte: Elaborado pelo autor (2024)

Vamos adicionar apenas 1 bit na etapa de quantização e refazer o gráfico. Então, subindo de 3 para 4 bits, teremos $2^4 = 16$ degraus ou níveis. Com a adição de 1 bits, conseguimos dobrar a quantidade de degraus, reduzindo significativamente a distância entre cada nível e cometendo, assim, um menor erro de arredondamento.

Veja os pontos em vermelho originais quase em cima dos pontos verdes e os demais pontos verdes já bateram com os vermelhos originais, não precisando de serem arredondados. Note que a crista da senoide voltou a ser mais redonda e próxima da original, e também menos achatada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

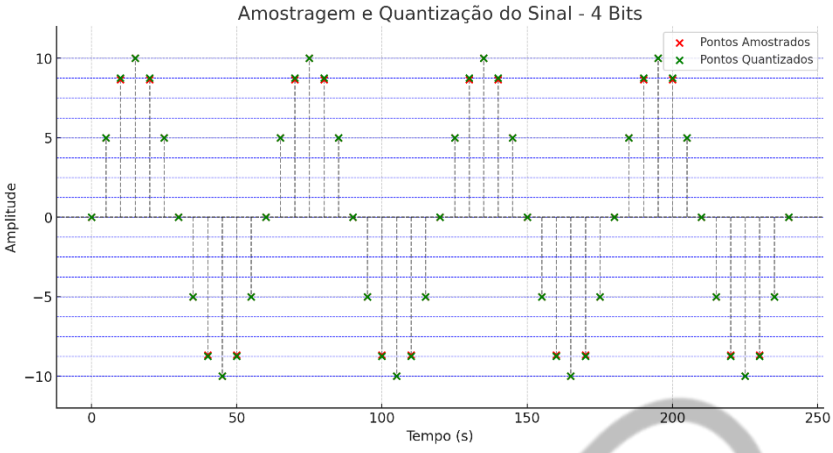

Figura 11 - Quantização usando 4 bits de resolução
Fonte: Elaborado pelo autor (2024)

Mais uma simulação, iremos adicionar mais 2 bits na quantização. Ao invés de 4, vamos colocar 6 bits. Assim, teremos 2^6 = 64 níveis. O gráfico resultante será o seguinte. Note que agora temos uma senoide mais próxima da perfeita na memória do ESP32 com apenas 6 bits de quantização. Mas ainda assim tem o erro de quantização nos picos mais altos da senoide, pois os pontos verdes não estão batendo com os vermelhos. Lembrando que onde não há vermelho no meio da onda é sinal que a quantização coincidentemente bateu no respectivo pico sem precisar arredondar.

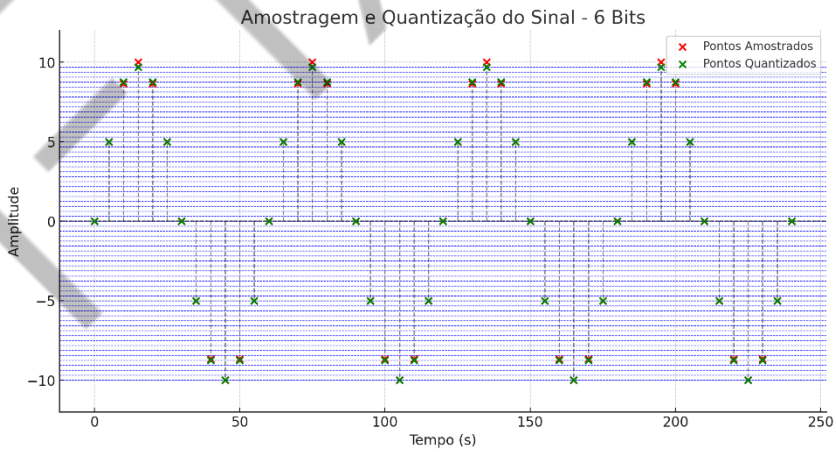

Figura 12 - Etapa de quantização usando 6 bits
Fonte: Elaborado pelo autor (2024)

Por fim, vamos colocar 8 bits de resolução. O gráfico resultado é o que mostra a figura “Etapa da quantização usando 8 bits de resolução”. A senoide está quase perfeita na memória do ESP32 porque os pontos verdes e vermelhos quase se encontram em todos os picos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

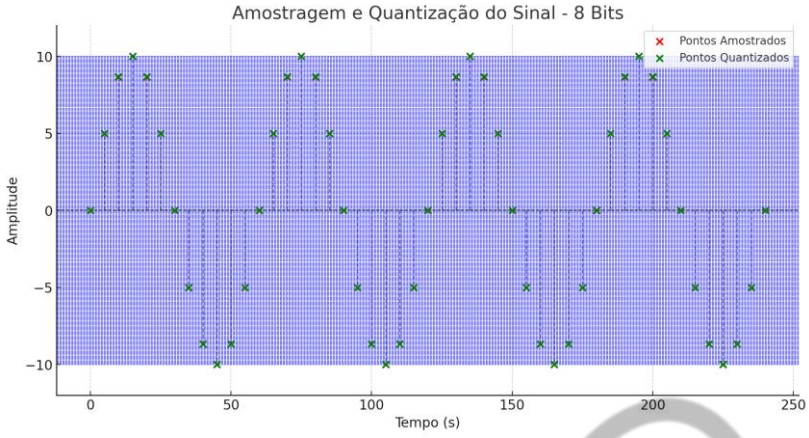

Figura 13 - Etapa da quantização usando 8 bits de resolução
Fonte: Elaborado pelo autor (2024)

Portanto, a quantização é uma etapa essencial tanto na coleta de dados com o ADC do ESP32 quanto na otimização de modelos de IA para sistemas embarcados. Ela permite a integração eficiente entre o mundo físico e os sistemas digitais, garantindo que projetos de Inteligência Artificial possam operar com precisão e eficiência em dispositivos de baixo custo e alta versatilidade, como o ESP32.

Como foi mencionado no início dessa seção, o ESP32 possui 12 bits de quantização no seu ADC que dão 4.096 degraus ou níveis. Graficamente, ficaria assim o sinal senoidal (figura a seguir).

Observação: o fundo azul claro da figura não é um simples fundo, e sim, 4.096 traços horizontais entre os valores -10 e +10 que de tão próximos um do outro, parece um fundo sólido.

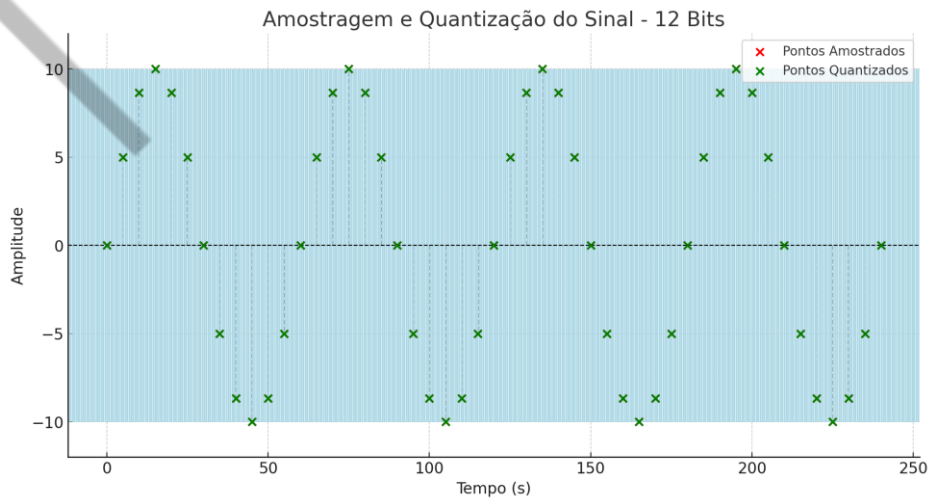

Figura 14 - Etapa de quantização usando 12 bits de resolução
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Todos os pontos estão verdes agora, indicando que o ESP32 teria considerado fielmente todos os picos da nossa senoide original entre -10 e +10 de amplitude, sem nenhum erro de quantização considerável. Esse seria um ótimo resultado como quantização.

2.1.3 Codificação

A codificação é a etapa final do processo de digitalização de sinais analógicos. Após a amostragem (que seleciona valores em intervalos de tempo regulares) e a quantização (que aproxima os valores analógicos para níveis discretos), a codificação converte esses níveis discretos em representações binárias (0s e 1s). Esses números binários podem ser processados, armazenados ou transmitidos por sistemas digitais, como o ESP32.

A codificação é essencial porque é nesse momento que o sinal digitalizado se torna compreensível para dispositivos eletrônicos digitais. Cada nível de quantização é associado a um código binário único, cujo número de bits depende da resolução do conversor analógico-digital (ADC). Por exemplo:

- Em um ADC com 3 bits (8 níveis), os níveis seriam codificados de 000 a 111.
- Em um ADC com 12 bits (4096 níveis), os níveis seriam codificados de 000000000000 a 111111111111.

O ADC do ESP32 suporta uma resolução máxima de 12 bits como já foi dito. Isso significa que ele pode quantizar um sinal analógico em até 4096 níveis discretos (2^{12} = 4096), e cada nível é codificado em uma sequência de 12 bits. Essa alta resolução permite ao ESP32 capturar sinais analógicos com boa precisão, tornando-o ideal para aplicações que exigem medições detalhadas, como em sensores de temperatura, pressão ou luminosidade.

No entanto, a resolução pode ser ajustada para valores menores, como 9, 10 ou 11 bits, dependendo das necessidades do projeto, o que pode influenciar o número de níveis de quantização e a precisão do sinal digitalizado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em resumo, o ESP32 utiliza codificação de até 12 bits no processo de digitalização, garantindo que os sinais analógicos sejam representados digitalmente com alta precisão.

E para entender como o ESP32 ou outro microcontrolador qualquer faz para finalizar o processo de digitalização de sinais analógicos, imagine que cada amostra do sinal senoidal é guardada em um endereço de memória de 12 divisórias por ser de 12 bits.

Como a contagem dos pesos dos bits começa em 0, ele vai até o peso 11 que equivale a 2048 em peso. Isso faz total sentido porque como temos na quantidade de degraus, metade do sinal positivo, e a outra metade na parte negativa, teremos então até 2048 para o positivo e 2048 para a parte negativa.

Então, calculando o valor do degrau na escala de -10 até 0 e depois +10.

Passo da quantização = 20 / 4096 = 0,00488. Isso significa que a cada 0,00488 numa escala de -10 0 +10 (20 no total) de amplitude do sinal senoidal, teremos um degrau no quantizador de 0,00488 em 0,00488.

Esse entendimento é importante para você acompanhar como o codificador finaliza o processo.

Veja a tabela a seguir:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

			POSIÇÕES DOS BITS NA MEMÓRIA DO ESP32											
PESOS →			2048	1024	512	256	128	64	32	16	8	4	2	1
Amostra original	Nível do quantizador de 12bits	Nível Codificador	11	10	9	8	7	6	5	4	3	2	1	0
-10	0	Salva	0	0	0	0	0	0	0	0	0	0	0	0
...	1	Não salva	0	0	0	0	0	0	0	0	0	0	0	1
...	2	Não salva	0	0	0	0	0	0	0	0	0	0	1	0
...	3	Não salva	0	0	0	0	0	0	0	0	0	1	1	0
...	4	Não salva	0	0	0	0	0	0	0	0	0	1	0	0
...	...	Não salva												
-7,5	512	Salva	0	0	1	0	0	0	0	0	0	0	0	0
...	...	Não salva												
-5	1024	Salva	0	1	0	0	0	0	0	0	0	0	0	0
...	...	Não salva												
0	2048	Salva	1	0	0	0	0	0	0	0	0	0	0	0
...	...	Não salva												
+5	3072	Salva	1	1	0	0	0	0	0	0	0	0	0	0
...	...	Não salva												
+7,5	3584	Salva	1	1	1	0	0	0	0	0	0	0	0	0
...	...	Não salva												
+10	4095	Salva	1	1	1	1	1	1	1	1	1	1	1	1

Tabela 1 - Explicação de como o codificador trabalha para codificar um dado pico
Fonte: Elaborado pelo autor (2024)

Então, o codificador vai salvar ou considerar os valores binários exatamente onde tem amostra original, que são os seguintes:

- Quando bater a amplitude -10, o valor codificado 000000000000 (12 bits) será salvo.
- Quando bater a amplitude -7,5, o valor codificado 001000000000 (12 bits) será salvo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Quando bater -5, o valor codificado 010000000000 (12 bits) será salvo.
- Quando bater em 0, o valor codificado 1000000000000 será salvo.
- Quando bater em +5, o valor considerado 110000000000 será salvo.
- Quando bater em +7,5, o valor considerado 111000000000 será salvo.
- Quando bater +10, o valor salvo será 111111111111.

Onde tem “não salva” na tabela, significa que ignoramos os valores dos intervalos entre as amostras e usamos reticências para indicar isso.

2.2 Códigos e Aplicações para ADC

Nesta seção, vamos explorar alguns exemplos de código para ESP32 e entender como funciona o hardware que adota toda a teoria de digitalização de sinais analógicos. E ainda, vamos mostrar o circuito usando o Wokwi.com.

2.2.1 Primeiro exemplo para ADC

Primeiro ponto é entender quais os pinos do ESP32 suportam o ADC. O ESP32 possui vários pinos ADC, organizados em dois ADCs internos (ADC1 e ADC2), cada um com múltiplos canais. Esses pinos permitem a leitura de sinais analógicos no intervalo de 0V a 3.3V, com uma resolução de até 12 bits (4096 níveis).

- **ADC1:**
  - GPIO 32 (ADC1_CH4);
  - GPIO 33 (ADC1_CH5);
  - GPIO 34 (ADC1_CH6);
  - GPIO 35 (ADC1_CH7);
  - GPIO 36 (ADC1_CH0, também chamado de VP);
  - GPIO 37 (ADC1_CH1);
  - GPIO 38 (ADC1_CH2);

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

  - GPIO 39 (ADC1_CH3, também chamado de VN).
- **ADC2:**
  - GPIO 0 (ADC2_CH1);
  - GPIO 2 (ADC2_CH2);
  - GPIO 4 (ADC2_CH0);
  - GPIO 12 (ADC2_CH5);
  - GPIO 13 (ADC2_CH4);
  - GPIO 14 (ADC2_CH6);
  - GPIO 15 (ADC2_CH3);
  - GPIO 25 (ADC2_CH8);
  - GPIO 26 (ADC2_CH9);
  - GPIO 27 (ADC2_CH7).

Quais as diferenças entre ADC1 e ADC2? O ADC1 pode ser usado em qualquer aplicação, mesmo durante o uso de Wi-Fi. É recomendado para leituras estáveis, especialmente quando o Wi-Fi está ativo. O ADC2 compartilha recursos com o módulo Wi-Fi do ESP32 e, por isso, não pode ser utilizado para leitura de ADC quando o Wi-Fi está ativo.

Para ter o efeito desejado, temos que inserir um sinal analógico no ESP32 para explorar os potenciais do recurso de ADC. A principal função para usar o recurso ADC é o analogRead(). Além disso, não são todos os pinos do ESP32 que podem atuar como ADC.

Esse sinal analógico virá de um potenciômetro que varia a sua resistência interna ao girar o pino central. E, ao variar a resistência, varia a sua queda de tensão simulando um sinal analógico de infinitas possibilidades de valores.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

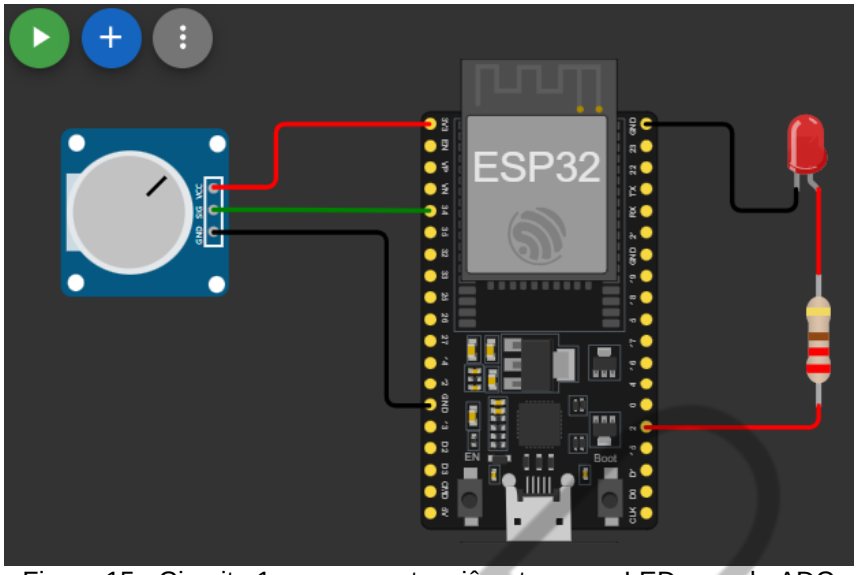

Figura 15 - Circuito 1 com um potenciômetro e um LED usando ADC
Fonte: Elaborado pelo autor (2024)

O circuito anterior está sendo controlado pelo código a seguir. A sua função é monitorar o sinal analógico de 3,3V inserido através do potenciômetro e, quando atinge um limite de 2,0V, imprime-se a mensagem “Alerta! Limite ultrapassado!”.

Você pode copiar e colar esse código em um projeto vazio no Wokwi utilizando o ESP32 e montar o circuito da figura anterior. O resistor do LED vermelho é de 220Ω.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#define SENSOR_PIN 34  // Pino do ADC
#define LED_PIN 2      // Pino do LED
const float LIMITE = 2.0;  // Limite em volts

void setup() {
  pinMode(LED_PIN, OUTPUT);  // Configura o LED como saída
  Serial.begin(115200);      // Inicia a comunicação serial
}

void loop() {
  int valorADC = analogRead(SENSOR_PIN);
  float tensao = (valorADC * 3.3) / 4095;

  Serial.print("Tensão: ");
  Serial.print(tensao);
  Serial.println(" V");

  if (tensao > LIMITE) {
    digitalWrite(LED_PIN, HIGH);  // Liga o LED
    Serial.println("Alerta! Limite ultrapassado!");
  } else {
    digitalWrite(LED_PIN, LOW);  // Desliga o LED
  }

  delay(200);  // Atraso de 200 ms
}
```

Código-fonte 1 - Exemplo de como ADC trabalha no ESP32
Fonte: Elaborado pelo autor (2024)

O resultado da simulação é mostrado no Monitor Serial com as impressões do nível da tensão e do alerta quando o algoritmo permitir.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

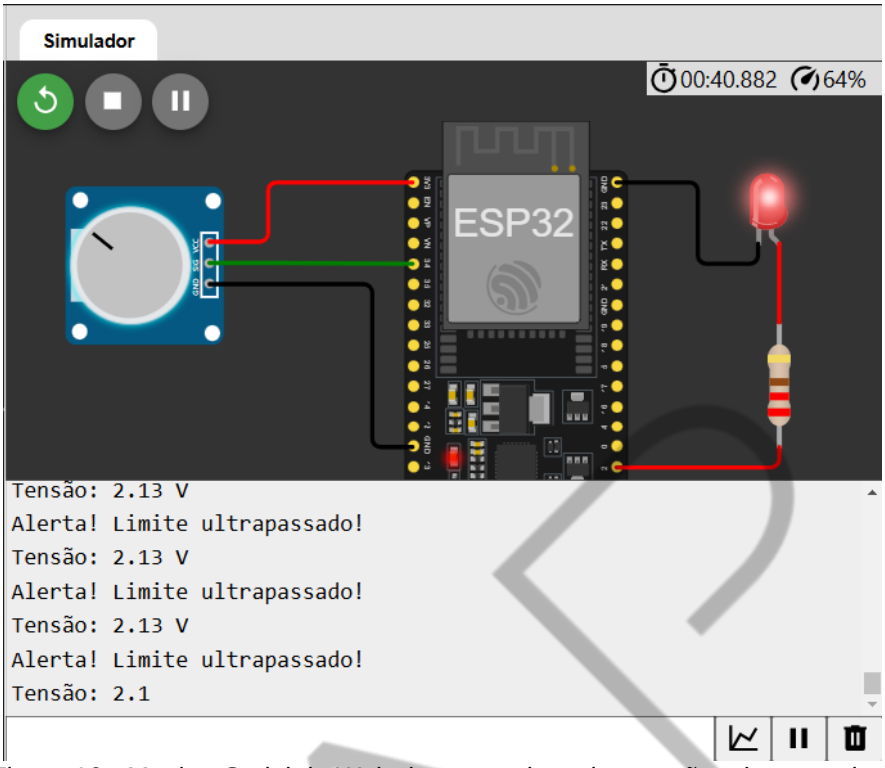

Figura 16 - Monitor Serial do Wokwi mostrando as impressões do exemplo 1
Fonte: Elaborado pelo autor (2024)

Para que serve esse primeiro exemplo? Considere que no lugar do potenciômetro, pode ser qualquer sensor analógico. Imagine desenvolver um sistema que monitora a umidade do solo em uma fazenda utilizando o ESP32 e um sensor de umidade. Os dados coletados são enviados a uma plataforma de IA na nuvem para análise preditiva, enquanto o ESP32 localmente ativa alarmes e notificações simples. O LED indicaria baixa umidade.

2.2.2 Segundo exemplo de ADC

Partindo para o segundo exemplo de código usando ADC, temos uma sugestão mais elaborada. Ele tira a média móvel entre 10 valores, uma importante técnica usada para sistemas de IA.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#define SENSOR_PIN 34
#define LED_PIN 2
const int TAMANHO_JANELA = 10;  // Tamanho da janela para a média
móvel
float leituras[TAMANHO_JANELA];
int indice = 0;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
  for (int i = 0; i < TAMANHO_JANELA; i++) {
    leituras[i] = 0.0;
  }
}

void loop() {
  // Lê o sensor
  int valorADC = analogRead(SENSOR_PIN);
  float tensao = (valorADC * 3.3) / 4095;

  // Atualiza a janela de leituras
  leituras[indice] = tensao;
  indice = (indice + 1) % TAMANHO_JANELA;

  // Calcula a média móvel
  float soma = 0.0;
  for (int i = 0; i < TAMANHO_JANELA; i++) {
    soma += leituras[i];
  }
  float media = soma / TAMANHO_JANELA;

  // Tomada de decisão com base na média
  Serial.print("Tensão atual: ");
  Serial.print(tensao);
  Serial.print(" V | Média: ");
  Serial.print(media);
  Serial.println(" V");

  if (media > 2.5) {
    digitalWrite(LED_PIN, HIGH);  // Liga o LED se a média for
maior que 2.5V
    Serial.println("Decisão: Alerta! Média alta.");
  } else {
    digitalWrite(LED_PIN, LOW);  // Desliga o LED
    Serial.println("Decisão: Normal.");
  }

  delay(200);  // Atraso de 200 ms
}
```

Código-fonte 2 - Segundo exemplo de ADC, mas agora usando média móvel
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No código, você encontrará a impressão da tensão atual e da média entre 10 leituras de dados.

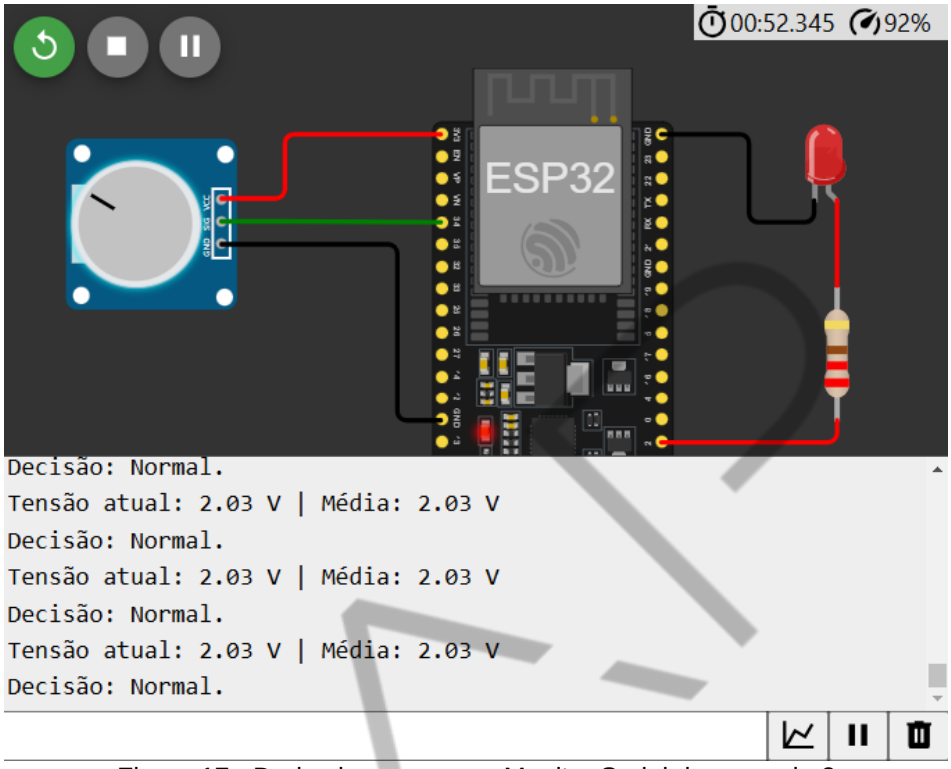

Figura 17 - Dados impressos no Monitor Serial do exemplo 2
Fonte: Elaborado pelo autor (2024)

A função desse segundo exemplo pode ser uma solução para carros autônomos ou uma empresa que deseja reduzir seus custos com energia. Um sistema de IA precisa da coleta de dados de energia para ser treinada depois. Os dados coletados deverão ser carregados no Python, por exemplo, onde o modelo deverá ser criado.

A média móvel é uma técnica para eliminar os *outliers*, pois ela suaviza as diferenças entre as amostras, eliminando flutuações rápidas. Isso pode melhorar a precisão dos dados quando há flutuações indesejadas como a presença de ruídos nos arredores dos sensores.

2.2.3 Terceiro exemplo de ADC

Um terceiro exemplo mais elaborado é um sistema de saúde que prevê se um paciente está com risco de saúde ou se está dentro dos padrões normais. Esse

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

sistema utiliza o ESP32 para coletar dados de temperatura e batimento cardíaco e aplica nesses dados em modelo do Tensorflow que deverá ser treinado pelo desenvolvedor. As etapas desse projeto são as seguintes:

1. O **ESP32 coleta dados** de sensores de temperatura e frequência cardíaca e classifica por meio de uma supervisão em “risco” e “saúde”, de acordo com os padrões médicos já conhecidos. Por exemplo, uma temperatura corporal entre **36.1 °C e 37.2 °C** e uma frequência cardíaca entre **60 e 100 BPM** são consideradas normais para adultos saudáveis em repouso, marcados como **saúde ou 1**. Qualquer desvio desses intervalos pode ser rotulado como **risco ou 0.**
2. **Modelo é treinado no Python**. Durante o treinamento, o modelo aprende a **associar padrões nas entradas (temperatura e frequência cardíaca) aos rótulos fornecidos (1 ou 0)**. A lógica do modelo é:
    a. Ajustar os pesos internos com base no erro entre a previsão (saída do modelo) e o rótulo real.
    b. Repetir o processo até minimizar o erro.
    c. Após o treinamento, o modelo pode prever se uma nova amostra de temperatura e de batimento representa **saúde (1)** ou **risco (0)** com base nos padrões aprendidos.
3. **Outro código de teste é executado em Python**. Nessa etapa, um outro código em Python recebe manualmente os dados de temperatura e batimento cardíaco, onde ele resultará em tempo real, a probabilidade do paciente estar saudável ou em risco.

Antes de começar a programar o projeto, veja alguns exemplos de rótulos do tipo “saúde” que será o UM e o “risco” que será representado pelo ZERO.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Temperatura (°C)	Frequência Cardíaca (BPM)	Condição	Rótulo
36.5	70	Dentro dos limites normais	1
39.0	100	Temperatura elevada (febre)	0
35.5	60	Limite inferior normal	1
40.0	120	Alta febre e taquicardia	0
37.0	80	Dentro dos limites normais	1
36.8	50	Bradicardia (frequência baixa)	0

Tabela 2 - Exemplo de critérios de rotulagem que serão feitas pelo ESP32
Fonte: Elaborado pelo autor (2024)

Quem define os rótulos no ESP32? Os rótulos são definidos por você, o desenvolvedor, com base nos critérios estabelecidos. Para uma amostra de **36.5 °C e 70 BPM**, o desenvolvedor observa que ambos os valores estão dentro dos limites normais. Assim, a amostra é rotulada como **1 (saúde)**. Para uma amostra de **39.0 °C e 100 BPM**, o desenvolvedor identifica que a temperatura está elevada (febre). Assim, a amostra é rotulada como **0 (risco)**.

E qual o papel do modelo durante o treinamento? Durante o treinamento, o modelo aprende a **associar padrões nas entradas (temperatura e frequência cardíaca)** aos rótulos fornecidos. A lógica do modelo é:

1. Ajustar os pesos internos com base no erro entre a previsão (saída do modelo) e o rótulo real. Esse exemplo de modelo usa redes neurais simples.
2. Repetir o processo até minimizar o erro.

O código a seguir é o que faz a coleta de temperatura, batimento cardíaco e rotulação em risco ou saúde:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#define TEMP_SENSOR_PIN 34   // Pino ADC para o sensor de
temperatura
#define HR_SENSOR_PIN 35      // Pino ADC para o sensor de
frequência cardíaca

// Função para determinar o rótulo baseado nos critérios médicos
int classificarSaude(float temperatura, float freqCardiaca) {
 #define TEMP_SENSOR_PIN 34   // Pino ADC para o sensor de
temperatura
#define HR_SENSOR_PIN 35     // Pino ADC para o sensor de
frequência cardíaca
#define INTERVALO_LEITURA 1000  // Intervalo entre leituras (em
milissegundos)

unsigned long ultimoTempo = 0;  // Armazena o último tempo de
execução

// Função para determinar o rótulo baseado nos critérios médicos

int classificarSaude(float temperatura, float freqCardiaca) {
  // Verifica se os valores estão dentro dos limites normais
  if (temperatura >= 36.1 && temperatura <= 37.2 && freqCardiaca
>= 60 && freqCardiaca <= 100) {
    return 1;  // Saúde
  } else {
    return 0;  // Risco
  }
}

void setup() {
  Serial.begin(115200);  // Inicializa a comunicação serial
}

void loop() {
  // Verifica se é hora de realizar uma nova leitura
  unsigned long tempoAtual = millis();
  if (tempoAtual - ultimoTempo >= INTERVALO_LEITURA) {
     ultimoTempo = tempoAtual;  // Atualiza o tempo da última
leitura

    // Leitura dos sensores
     int tempADC = analogRead(TEMP_SENSOR_PIN);  // Leitura do
sensor de temperatura
     int hrADC = analogRead(HR_SENSOR_PIN);      // Leitura do
sensor de frequência cardíaca

    // Conversão para valores reais
    float temperatura = (tempADC * 3.3 / 4095) * 100;  // Exemplo:
0-100°C
    float freqCardiaca = (hrADC * 3.3 / 4095) * 180;   // Exemplo:
0-180 BPM
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    // Rotulação com base nos critérios médicos
    int rotulo = classificarSaude(temperatura, freqCardiaca);

    // Exibição dos dados e rótulo
    Serial.print("Temperatura: ");
    Serial.print(temperatura);
    Serial.print(" °C | Frequência Cardíaca: ");
    Serial.print(freqCardiaca);
    Serial.print(" BPM | Rotulo: ");
    if (rotulo == 1) {
      Serial.println("Saúde (1)");
    } else {
      Serial.println("Risco (0)");
    }
  }

  // Outras tarefas podem ser adicionadas aqui sem interferir no
tempo de leitura
}
```

Código-fonte 3 - Exemplo de uso de ADC para um sistema de saúde
Fonte: Elaborado pelo autor (2024)

Para esse código, vamos adotar dois potenciômetros fazendo o papel de sensores analógicos conectados ao ESP32. Portanto, você pode montar esse projeto no Wokwi.com. No projeto, existe um potenciômetro superior conectado no GPIO 34, enquanto o segundo potenciômetro está no GPIO 35. Ambos estão com os pinos VCC ligados no pino 3,3V e os pinos GND, no GND do ESP32.

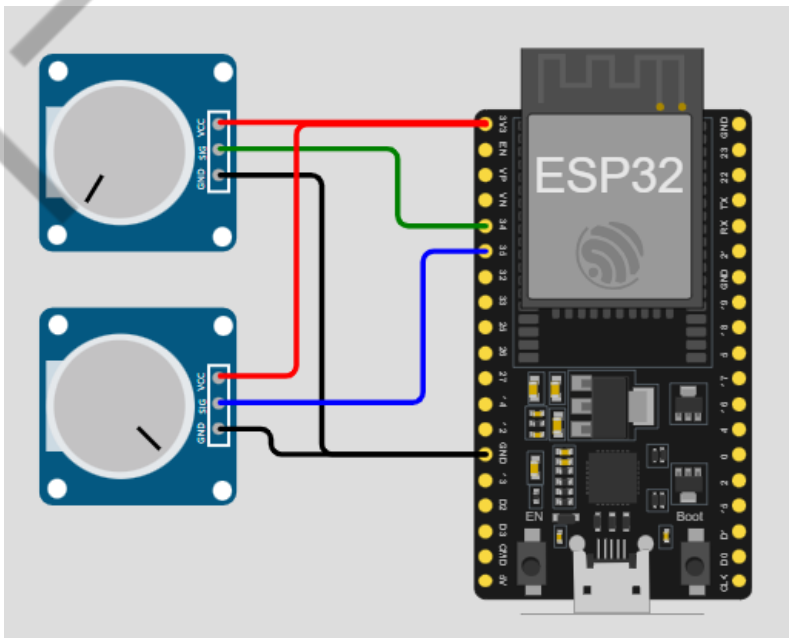

Figura 18 - Circuito do ESP32 coletando dados por meio de dois potenciômetros
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Após algumas dezenas de leitura dos dados, começaremos a etapa do treinamento do modelo em Tensorflow. Suponha que você tenha detectado os seguintes dados de temperatura e frequência cardíaca através de sensores analógicos do ESP32 e dentro do algoritmo do ESP32 já classificado como “1” e “0”. Esses dados você pode copiar do Monitor Serial do ESP32. Os dados são impressos no Monitor através de comandos Serial.println() do código C/C++, como já vimos em capítulos anteriores. Aqui estão algumas amostras, mas que na prática, serão necessários uma centena deles para ter melhor acurácia no modelo.

Temperatura (°C)	Frequência Cardíaca (BPM)	Rótulo
36.5	70	1
39.0	100	0
35.5	60	1
40.0	120	0
37.0	80	1
38.5	95	0
36.8	75	1

Tabela 3 - Exemplo de rotulação de “saúde” (1) e “risco” (1)
Fonte: Elaborado pelo autor (2024)

Ao passar esses dados para o Python, montaremos dois arrays (vetor) numpy, um com os dados de temperatura e batimento e outro com o rótulo saúde e risco.

```
import tensorflow as tf
import numpy as np
import os #para facilitar a encontrar o arquivo resultante
(modelo)

# Dados de entrada (temperatura e frequência cardíaca) e saída
(saúde)
# 1ª coluna: temperatura (°C), 2ª coluna: frequência cardíaca
(BPM)
X = np.array([
    [36.5, 70],
    [39.0, 100],
    [35.5, 60],
    [40.0, 120],
    [37.0, 80],
    [38.5, 95],
    [36.8, 75]
])

# Saídas correspondentes (saudável ou risco)
y = np.array([1, 0, 1, 0, 1, 0, 1])  # 1 = saudável, 0 = risco
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Normalização dos dados
X = X / np.array([40.0, 120.0])  # Escala máxima para
normalização

# Construção do modelo
model = tf.keras.Sequential([
              tf.keras.layers.Dense(16,      activation='relu',
input_shape=(2,)),  # Camada oculta com 16 neurônios
                                        tf.keras.layers.Dense(8,
activation='relu'),                       # Camada oculta com 8
neurônios
                                        tf.keras.layers.Dense(1,
activation='sigmoid')                  # Saída (probabilidade de
saúde)
])

# Compilação do modelo
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Treinamento do modelo
model.fit(X, y, epochs=100, batch_size=4)

# Avaliação do modelo
loss, accuracy = model.evaluate(X, y)
print(f"Loss: {loss}, Accuracy: {accuracy}")

# Salvar o modelo
model.save("saude_model.keras")
print(os.listdir('.')) # Lista todos os arquivos no diretório
atual, incluindo saude_model.keras
```

Código-fonte 4 - Código de treino do modelo de saúde e risco
Fonte: Elaborado pelo autor (2024)

Rode esse código no Colab que vai funcionar. Após ter dado o play na célula do Colab, você pode encontrar o arquivo modelo com as dicas a seguir. Ele está na pasta *content*. Caso não esteja encontrando, precisa executar o código da célula primeiro e depois atualizar clicando no disco de retorno.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

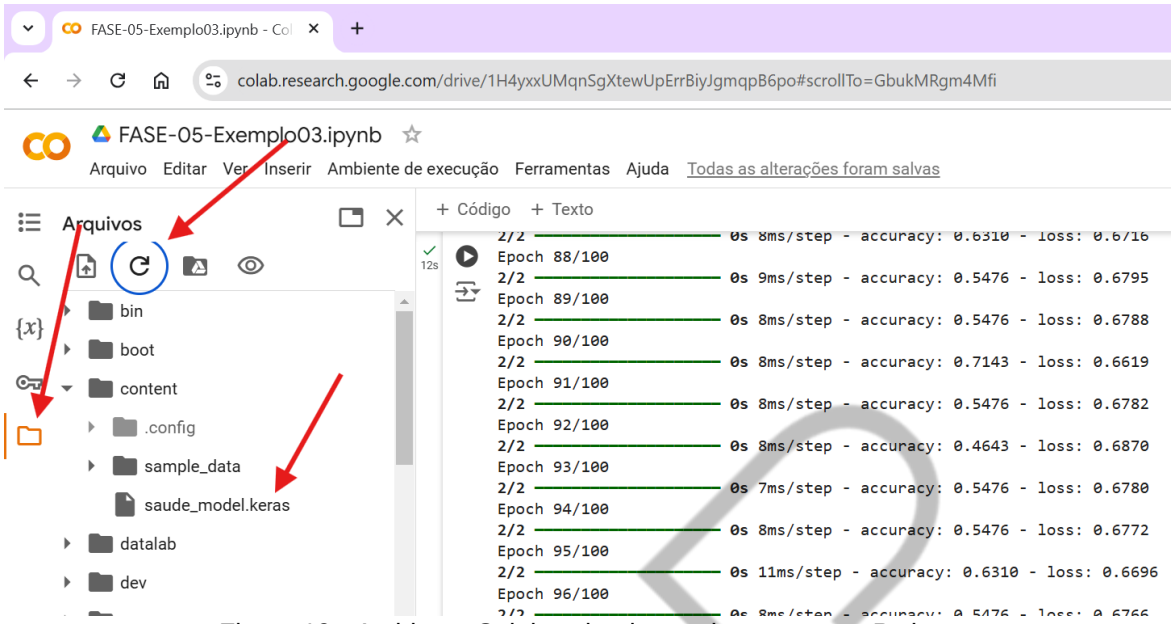

Figura 19 - Ambiente Colab rodando o treinamento em Python
Fonte: Elaborado pelo autor (2024)

Contudo, a ideia não é sair do Colab fazendo o download do arquivo *saude_model.keras*, e sim, rodar um outro programa onde você testará o seu modelo inserindo dados de temperatura e batimento de forma manual.

Insira uma nova célula de código no Colab abaixo do código de treinamento do modelo e acrescente esse código a seguir.

```
import tensorflow as tf
# Carregar o modelo treinado
model = tf.keras.models.load_model("saude_model.keras")

# Função para normalizar os dados (baseado no pré-processamento usado durante o treinamento)
def normalizar_dados(temperatura, freq_cardiaca):
    temp_norm = temperatura / 40.0  # Normaliza temperatura para escala 0-1
    freq_norm = freq_cardiaca / 120.0  # Normaliza frequência cardíaca para escala 0-1
    return [temp_norm, freq_norm]

# Função para classificar a entrada
def classificar_saude(temperatura, freq_cardiaca):
    # Normalizar os dados
        entrada_normalizada = normalizar_dados(temperatura, freq_cardiaca)
                                entrada_tensor = tf.convert_to_tensor([entrada_normalizada], dtype=tf.float32)

    # Fazer a predição
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    probabilidade = model.predict(entrada_tensor)[0][0]
    rotulo = "Saúde" if probabilidade >= 0.5 else "Risco"

    return rotulo, probabilidade

# Loop para receber dados do ESP32 via input manual
print("Insira os dados de temperatura e frequência cardíaca do
ESP32:")
while True:
    try:
        # Recebe dados do usuário
        temperatura = float(input("Temperatura (°C): "))
        freq_cardiaca = float(input("Frequência Cardíaca (BPM):
"))

        # Classificar os dados
         rotulo, probabilidade = classificar_saude(temperatura,
freq_cardiaca)

        # Exibir o resultado
        print(f"\nResultado da Classificação:")
        print(f"Temperatura: {temperatura} °C")
        print(f"Frequência Cardíaca: {freq_cardiaca} BPM")
                                print(f"Classificação:      {rotulo}
({probabilidade*100:.2f}%)\n")

    except ValueError:
        print("Por favor, insira valores numéricos válidos para
temperatura e frequência cardíaca.")
    except KeyboardInterrupt:
        print("\nSaindo do programa.")
        break
```

Código-fonte 5 - Teste do modelo inserindo dados manualmente
Fonte: Elaborado pelo autor (2024)

Esse modelo utiliza o **Keras**, uma biblioteca de alto nível para a construção e treinamento de modelos de aprendizado de máquina, integrada ao **TensorFlow**. Ela é usada para definir, treinar e salvar o modelo de **classificação de saúde ou risco**.

E, observando o código, você vai encontrar o "**binary_crossentropy**" que é uma **função de perda** usada em problemas de classificação binária, onde as saídas podem ser apenas duas classes, como **0** (risco) e **1** (saúde) no contexto deste projeto.

O que é uma função de perda? A função de perda mede o erro entre as previsões do modelo e os valores reais (rótulos). Durante o treinamento, o modelo ajusta seus pesos para minimizar esse erro.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por que Binary Crossentropy? Porque é utilizada especificamente para problemas de **classificação binária**. Ela é ideal para modelos que geram uma saída de probabilidade (0 a 1) em cada previsão, como quando usamos a função de ativação **sigmoid** na camada de saída. A função **sigmoid** na camada de saída do modelo retorna uma probabilidade entre 0 e 1.

Na prática, é possível transmitir os dados impressos no Monitor Serial do ESP32 para a Internet. Assim, o seu ESP32 desempenha o papel de coletor de dados, enquanto o Python desempenha o papel de classificação. Esse sistema integra uma solução de IA com IoT.

2.3 Conversor DAC

Um Digital Analog Converter (DAC) ou conversor de Digital para Analógico é um conjunto de componentes eletrônicos integrados que converte um sinal digital (binário) em um sinal analógico contínuo.

Afinal, nós somos humanos e analógicos, enquanto as máquinas são digitais. Portanto, para conseguirmos compreender as máquinas, precisamos retornar o sinal que está dentro delas para o mundo dos humanos. Isto é, converter o sinal digital em analógico.

No ESP32, isso não é feito por qualquer pino. E ainda, no ESP32, as saídas analógicas são níveis de tensão de diversos valores. Esse nível de tensão variado pode ser um áudio, um controle de brilho de um LED, um batimento cardíaco, um nível de pressão ou nível de água etc. Mas tudo será uma tensão em volts, compreendida entre 0 e 3,3V.

- O ESP32 possui **dois pinos DAC** com resolução de **8 bits** (isto é, 256 níveis, valores de 0 a 255). O número de 256 é porque a base de qualquer sistema digital é o 2 (2 de binário) e o número de bits em jogo é igual a 8. Portanto, $2^8 = 256$.
    - GPIO 25 (DAC1);
    - GPIO 26 (DAC2).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A função **dacWrite(pin, value)** converte um valor digital (número inteiro) em um **sinal analógico contínuo** (tensão) e o envia para o pino DAC do ESP32.

- **pin:** o pino DAC onde o sinal analógico será gerado. O ESP32 tem dois pinos DAC disponíveis:
  - GPIO 25 (DAC1).
  - GPIO 26 (DAC2).
- **value:** um número inteiro no intervalo de 0 a 255 (porque o DAC do ESP32 possui sempre 8 bits de resolução), representando o nível da tensão analógica a ser gerada.
  - **0** gera 0V (tensão mínima)
  - **255** gera a tensão máxima (geralmente 3.3V).

O resultado é um valor digital convertido internamente em um nível de tensão analógica correspondente e enviado ao pino.

Como o DAC converte o valor? Como já dito, o DAC do ESP32 tem resolução de **8 bits**, ou seja, pode representar **256 níveis diferentes de tensão** no intervalo de 0 a 3.3V. A relação entre o valor e a tensão gerada é dada pela seguinte equação.

$$Saída = \frac{valor}{255} . V_{max}$$

Onde:

- Saída = Tensão de saída analógica (em volts).
- V_{max} = 3,3, que é a tensão de alimentação do ESP32.

Exemplo de conversão:

- Valor digital 0: gera 0V.
- Valor digital 128: 1.65V (aproximadamente metade da tensão máxima).
- Valor digital 255: gera 3.3V

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3.1 Exemplo de DAC

Para demonstrar o funcionamento do DAC no ESP32, que é o retorno do sinal analógico ao seu formato original, podemos usar esse código como exemplo:

```
#define DAC_PIN 25  // Pino DAC1 no ESP32
#define NUM_SAMPLES 100  // Número de amostras na senoide
#define FREQUENCY 1  // Frequência da senoide em Hz

float senoide[NUM_SAMPLES];

void setup() {
  Serial.begin(115200);  // Inicializa a comunicação serial

  // Calcula os valores da senoide
  for (int i = 0; i < NUM_SAMPLES; i++) {
    senoide[i] = 200 * (sin(2 * PI * i / NUM_SAMPLES) + 1);  //
Escala para 0-255
  }
}

void loop() {
  for (int i = 0; i < NUM_SAMPLES; i++) {
    int value = (int)senoide[i];
    dacWrite(DAC_PIN, value);  // Envia o valor ao DAC
    Serial.println(value);  // Exibe o valor enviado no console
    delayMicroseconds(1000000 / (NUM_SAMPLES * FREQUENCY));
  }
}
```

Código-fonte 6 - Exemplo de código para reconstituir o sinal senoidal que estava digital
Fonte: Elaborado pelo autor (2024)

No ESP32, rode esse último programa e clique para visualizar o monitor em forma de gráfico.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

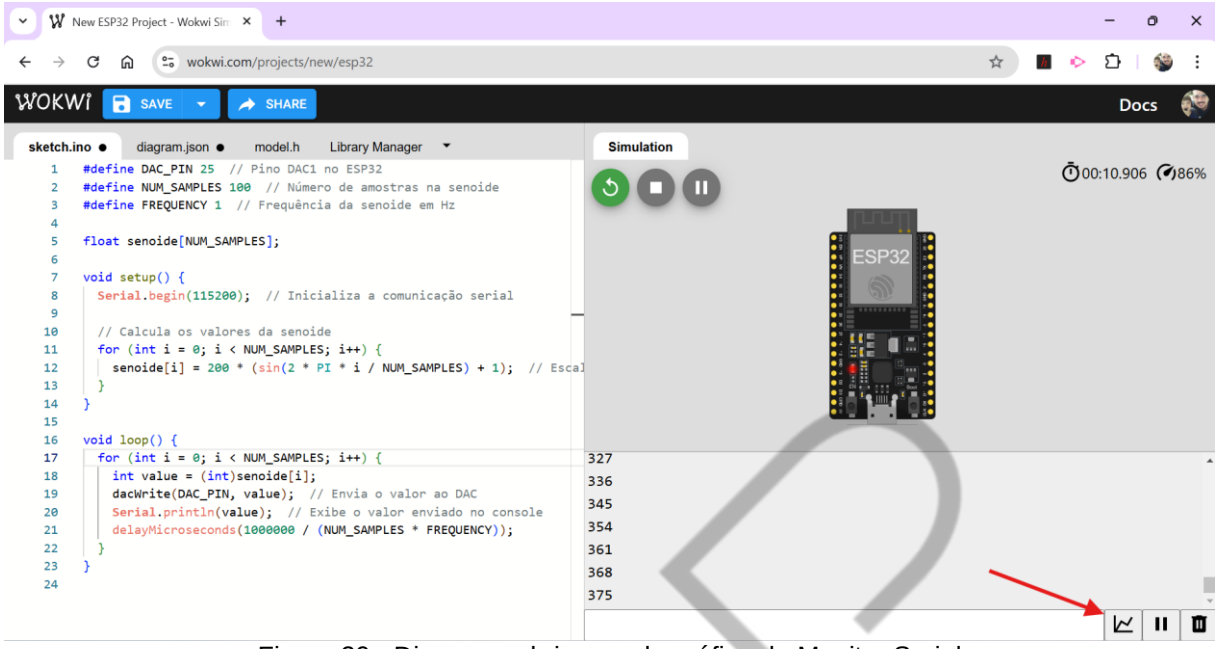

Figura 20 - Dica para abrir o modo gráfico do Monitor Serial
Fonte: Elaborado pelo autor (2024)

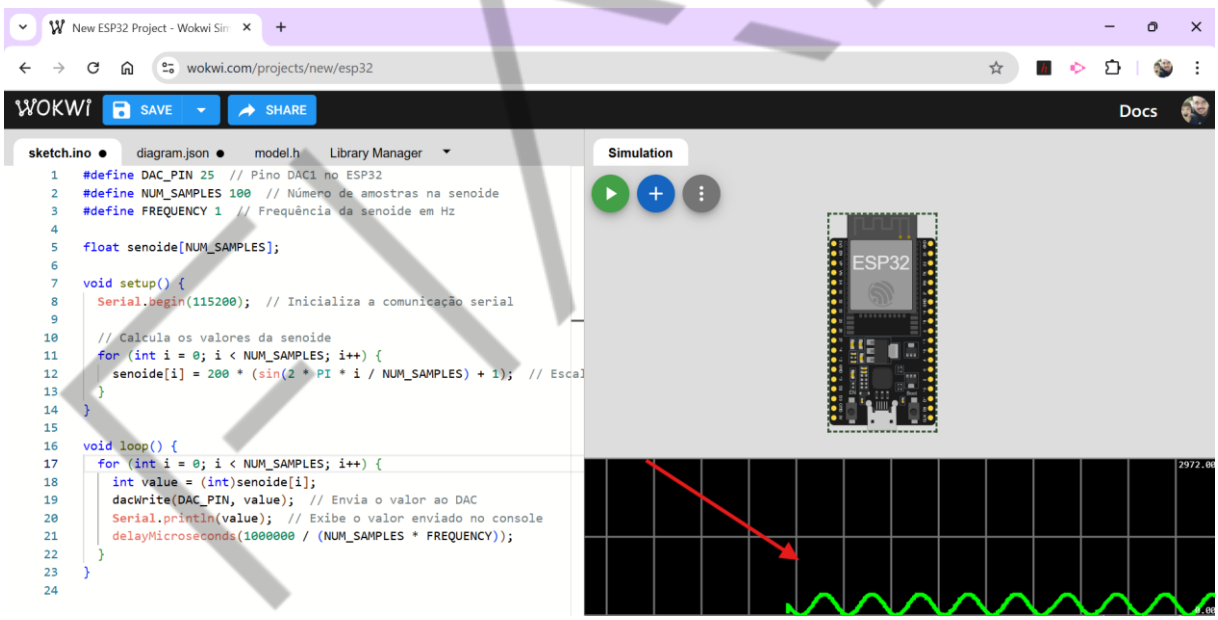

Figura 21 - Senoide sendo reproduzida pelo GPIO 25
Fonte: Elaborado pelo autor (2024)

Portanto, os **pinos** do ESP32 permitem a geração de sinais analógicos. Embora a resolução de 8 bits seja limitada, é possível utilizá-los de algumas formas em soluções de IA. Vejamos algumas:

Geração de sinais analógicos para controle, onde o DAC pode ser utilizado para controlar dispositivos analógicos, como motores, LEDs de intensidade variável ou atuadores, com base nas decisões tomadas por um modelo de IA. Em um sistema

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

de IA para agricultura, o DAC pode controlar a potência de um motor de bomba de água dependendo dos níveis de umidade do solo analisados.

Geração de áudio inteligente, onde o DAC pode gerar sinais de áudio, como alarmes ou notificações baseadas em decisões de IA. Um sistema de monitoramento de saúde pode gerar tons sonoros. Por exemplo, batimento cardíaco simulado para alertar sobre uma condição de risco detectada por um modelo de IA.

Gerar dados de teste para treinamento de modelos de IA, no qual o DAC pode ser usado para simular sinais analógicos de sensores, como sinais de eletrocardiograma (ECG), para validar modelos de IA em dispositivos médicos. Por exemplo, simular uma onda de ECG e verificar como o modelo de IA processa e interpreta os sinais.

Um último exemplo de código, pense em um **sistema de notificação de saúde**, onde um modelo de IA analisa batimentos cardíacos de um sensor conectado ao ESP32. O DAC gera tons audíveis diferentes:

- Tom normal para condições saudáveis.
- Tom de alerta em caso de anormalidades detectadas.

Dica: por questões de limitações da plataforma Wokwi.com e do Colab, ambos não suportam a geração de áudio no computador, vamos fazer uma simulação do DAC utilizando o Python e tons de áudio em forma de gráficos.

Rode o programa a seguir no Colab logo após o programa de treino saúde e risco.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Carregar o modelo treinado
model = tf.keras.models.load_model("saude_model.keras")

# Frequências para os tons (em Hz)
TONE_SAUDE = 440  # Frequência para saúde (A4)
TONE_RISCO = 880  # Frequência para risco (A5)

# Configurações do áudio
SAMPLE_RATE = 44100  # Taxa de amostragem (Hz)
DURATION = 1  # Duração do som (segundos)

# Função para normalizar os dados (baseado no pré-processamento
usado durante o treinamento)
def normalizar_dados(temperatura, freq_cardiaca):
    temp_norm = temperatura / 40.0  # Normaliza temperatura para
escala 0-1
    freq_norm = freq_cardiaca / 120.0  # Normaliza frequência
cardíaca para escala 0-1
    return [temp_norm, freq_norm]

# Função para classificar a entrada
def classificar_saude(temperatura, freq_cardiaca):
    # Normalizar os dados
    entrada_normalizada = normalizar_dados(temperatura,
freq_cardiaca)
    entrada_tensor =
tf.convert_to_tensor([entrada_normalizada], dtype=tf.float32)

    # Fazer a predição
    probabilidade = model.predict(entrada_tensor)[0][0]
    rotulo = "Saúde" if probabilidade >= 0.5 else "Risco"

    return rotulo, probabilidade

# Função para simular a onda senoidal
def gerar_onda(frequencia, duracao, sample_rate):
    t = np.linspace(0, duracao, int(sample_rate * duracao),
endpoint=False)
    return t, 0.5 * np.sin(2 * np.pi * frequencia * t)  # Senoide
com amplitude 0.5

# Loop para receber dados e simular tons
print("Insira os dados de temperatura e frequência cardíaca do
ESP32:")
while True:
    try:
        # Recebe dados do usuário
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        temperatura = float(input("Temperatura (°C): "))
        freq_cardiaca = float(input("Frequência Cardíaca (BPM): "))

        # Classificar os dados
        rotulo, probabilidade = classificar_saude(temperatura, freq_cardiaca)

        # Exibir o resultado
        print(f"\nResultado da Classificação:")
        print(f"Temperatura: {temperatura} °C")
        print(f"Frequência Cardíaca: {freq_cardiaca} BPM")
        print(f"Classificação: {rotulo} ({probabilidade*100:.2f}%)\n")

        # Gerar a forma de onda correspondente
        if rotulo == "Saúde":
            print("Visualizando onda de Saúde...")
            t, onda = gerar_onda(TONE_SAUDE, DURATION, SAMPLE_RATE)
        else:
            print("Visualizando onda de Risco...")
            t, onda = gerar_onda(TONE_RISCO, DURATION, SAMPLE_RATE)

        # Visualizar a forma de onda
        plt.figure(figsize=(10, 4))
        plt.plot(t[:1000], onda[:1000], label=f"{rotulo} ({TONE_SAUDE if rotulo == 'Saúde' else TONE_RISCO} Hz)")
        plt.title("Visualização do Tom Gerado")
        plt.xlabel("Tempo (s)")
        plt.ylabel("Amplitude")
        plt.legend()
        plt.grid()
        plt.show()
        plt.close()  # Fecha o gráfico para continuar o loop

    except ValueError:
        print("Por favor, insira valores numéricos válidos para temperatura e frequência cardíaca.")
    except KeyboardInterrupt:
        print("\nSaindo do programa.")
        break
```

Código-fonte 7 - Exemplo de sistema de saúde com dois tons distintos, um para “saúde” e outro para “risco”

Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

A digitalização de sinais analógicos no ESP32 envolve as etapas de amostragem, quantização e codificação, transformando dados contínuos do ambiente em valores digitais compreensíveis.

A amostragem captura o sinal em intervalos regulares, enquanto a quantização o ajusta para níveis discretos baseados na resolução do conversor. Por fim, a codificação transforma esses níveis em sequências binárias armazenáveis.

Esses processos são essenciais para projetos de Inteligência Artificial, permitindo que sensores analógicos como os de temperatura, pressão, umidade, entre vários outros alimentem modelos inteligentes.

A precisão da digitalização depende da frequência de amostragem e da resolução do ADC. Com o teorema de Nyquist-Shannon, define-se a frequência mínima de amostragem necessária para preservar as características do sinal original. Em aplicações práticas, como monitoramento de saúde, erros de quantização podem ser minimizados aumentando os bits de resolução, garantindo maior fidelidade ao sinal analógico.

A aplicação do DAC no ESP32, por outro lado, converte dados digitais de volta para o mundo analógico. Isso é a base para nós humanos entendermos as máquinas, ou seja, para gerar sinais de saída, como sons ou tensões, que controlam dispositivos externos, áudio etc.

Um exemplo prático é o uso do DAC para gerar tons sonoros indicativos de “saúde” ou “risco”, com base na análise de dados médicos por IA.

Os códigos apresentados demonstram como integrar essas funções ao ESP32, desde a coleta de dados analógicos até sua interpretação e saída. A combinação de ADC e DAC em projetos de IA habilita sistemas interativos, como dispositivos médicos, agrícolas e industriais, que conectam o mundo físico ao digital de forma eficaz.

Assim, o ESP32, com suas capacidades de conversão e integração, é uma ferramenta poderosa para soluções de IA embarcada. Ele viabiliza projetos acessíveis

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

e robustos em setores variados, destacando-se por sua versatilidade e custo-benefício.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

DATASHEET, ESP32 Series. **Espressif Systems**, Shanghai, China. 2019.

GHEORGHE, A. C.; STAN, E.; UDROIU, I. Electricity Consumption Measurement System Using ESP32. **The Scientific Bulletin of Electrical Engineering Faculty**, v. 21, n. 2, p. 23-26, 2021.

PROAKIS, John G. **Digital signal processing: principles algorithms and applications**. Pearson Education India, 2001.

SMITH, Steven W. The Scientist and Engineer's Guide to Digital Signal Processing. **California Technical Pub**, 1997.

VERMESAN, Ovidiu; NAVA, Mario Diaz; DEBAILLIE, Björn (Ed.). **Embedded artificial intelligence: Devices, embedded systems, and industrial applications**. CRC Press, 2023.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com